

PatroniLongRunningTransactionDetected

Overview

This alert means that there are transactions older than 10 minutes running on the server listed in the alert.

This could be caused by many things, most often a large sidekiq job. The alert will tell you what endpoint is executing the long running transaction.

This is important as long running transactions can prevent Postgres from running routine vacuuming, which can lead to bloat and slowdowns of the database.

Long-running transactions are one of the main drivers for `pg_txid_xmin_age`, and can result in severe performance degradation if left unaddressed.

The recipient of this alert should investigate what the long running transaction is, and whether it is going to cause performance problems. In most cases we are going to want to cancel the transaction.

Services

- Patroni Service
- Team that owns the service: Production Engineering : Database Reliability

Metrics

- Long Running Alerts Dashboard
- This alert fires when we measure one or more transactions with an age greater than 9 minutes, AND we have been in this state for 1 minute - which gives a total of 10 minutes that a transaction can be active before alerting
- Under normal conditions this dashboard should show lists of transactions with ages less 1 minute. Occasionally, there will be transactions which have been running for longer, but very few should approach the threshold

Alert Behavior

- This alert will clear once the long running transactions are no longer active. The alert should only be silenced during an open Change Request
- This alert should be fairly rare, and usually indicates that there is a query that is not behaving as we expect.
- Previous Incidents for alert

Severities

- This alert is unlikely to be causing active customer issues, and is most likely an S4

- However, this alert could evolve into performance issues for all of GitLab.com
- Check the Patroni SLI Overview Dashboard to determine whether we are already experiencing performance issues

Verification

- Prometheus query
- Long running transactions dashboard

Recent changes

- Recent Patroni Service change issues
- Recent Patroni Change Requests
- This alert is likely to have been triggered by a recent deployment, rather than a database related change.
- If there is a deployment causing the issue, roll back the change that was deployed
- If a change request caused the problem, follow the rollback instructions in the Change Request.

Troubleshooting

Getting the current query

The first step is to figure out what the long-running transaction is doing. The alert will tell you which host to look at. From there, it is possible to get a `query`, `pid`, `client_addr`, and `client_port`:

```
iwiedler@patroni-main-2004-01-db-gprd.c.gitlab-production.internal:~$ sudo gitlab-psql
```

```
gitlabhq_production=# \x
Expanded display is on.
```

```
gitlabhq_production=# \pset pager 0
Pager usage is off.
```

```
gitlabhq_production=# select *,
    now(),
    now() - query_start as query_age,
    now() - xact_start as xact_age
from pg_stat_activity
where
    state != 'idle'
    and backend_type != 'autovacuum worker'
    and xact_start < now() - '60 seconds'::interval
order by now() - xact_start desc nulls last
```

;

In the event that the alert has already cleared because the query in question has completed, or been canceled, it may still be possible to find the query in the Postgres `pg_stat_activity` logs. These transactions are dumped to Kibana every several minutes, so the same query may appear multiple times if it spans multiple dumps.

Getting the pgbouncer instance

Usually postgres connections go through a pgbouncer. The `client_addr` and `client_port` will tell you which one.

In this case it was `pgbouncer-01-db-gprd` as discovered via the `client_port` of 42792:

```
iwiedler@patroni-main-2004-01-db-gprd.c.gitlab-production.internal:~$ sudo netstat -tp | gr
tcp          0      229 patroni-main:postgresql pgbouncer-01-db-g:42792 ESTABLISHED 3889825/postgres
```

Resolving pgbouncer port to client

Now that we have the pgbouncer address and port, we can log into that pgbouncer box and get the actual client.

This can be done by first running `show sockets` on the pgbouncer admin console, and finding the metadata for the backend port:

```
iwiedler@pgbouncer-01-db-gprd.c.gitlab-production.internal:~$ sudo pgb-console -c 'show socke
type |      user      |      database      | state |      addr      | port | local_addr | local_port
S    | gitlab         | gitlabhq_production | sv_active | 10.220.21.101 | 5432 | 10.217.4.3 | 5432
```

Using this information we can then grab the `link` column. And join it against `show clients` to discover the actual client:

```
iwiedler@pgbouncer-01-db-gprd.c.gitlab-production.internal:~$ sudo pgb-console -c 'show clien
type |  user  |      database      | state |      addr      | port | local_addr | local_port
C    | gitlab | gitlabhq_production | active | 10.218.5.2     | 49266 | 10.217.4.5 | 49266
```

Double checking connections, and this tells us that it's `console-01-sv-gprd`:

```
iwiedler@pgbouncer-01-db-gprd.c.gitlab-production.internal:~$ sudo netstat -tp | grep 49266
tcp          0      35 10.217.4.5:6432          console-01-sv-gpr:49266 ESTABLISHED 19532/pgbouncer
```

Resolving client address and port to process and user

Now that we've confirmed it's a console user, we can look up which process on the console host holds that connection:

```
iwiedler@console-01-sv-gprd.c.gitlab-production.internal:~$ sudo ss -tp | grep 49266
```

```
ESTAB      0      194      10.218.5.2:49266      10.217.4.5:6432  users:((("ru
```

And we can feed the pid into `pstree` to see the process hierarchy:

```
systemd(1)  sshd(1369)  sshd(3225836)  sshd(3226021,arihant-rails)  bash(3226022)  script(3226
```

In this case it was user `arihant-rails` who was running a script via rails console.

Cancelling the query

In most cases, we will want to cancel the query. If it's a single long-running query, this can be done via `pg_cancel_backend` (passing in the pid):

```
gitlabhq_production=# select pg_cancel_backend(<pid>);
pg_cancel_backend
-----
t
```

However, if we are dealing with a long-running transaction consisting of many short-lived queries, it may be necessary to terminate the backend instead:

```
gitlabhq_production=# select pg_terminate_backend(<pid>);
pg_terminate_backend
-----
t
```

Possible Resolutions

- 2023-02-01: long postgres transaction was a long running transaction in a rails console session.
- 2021-08-18: Long running transaction on database was expected behavior as a foreign key was being added.
- 2023-10-21: Long Postgres transactions Vulnerabilities::MarkDroppedAsResolvedWorker was an application issue that warranted an InfraDev issue.
- Other incidents for this alert

Dependencies

- Background migrations
- Sidekiq jobs

Escalation

- If the recipient of this alert can't determine the cause of the long running transaction and correct it using the troubleshooting steps above, it may be necessary to escalate
- Slack channels where help is likely to be found: `#g_infra_database_reliability`

Definitions

- Link to the definition of this alert for review and tuning
- The main parameter that we can tune is the amount of time we allow before alerting. It is currently set to 10 minutes. We can also exclude certain types of transactions from alerting
- Link to edit this playbook
- Update the template used to format this playbook

Related Links

- Related alerts
- Previous Incidents involving this alert
- Wraparound status
- Handling unhealthy patroni replica
- Troubleshooting performance degradation
- Postgres Long Running Transaction